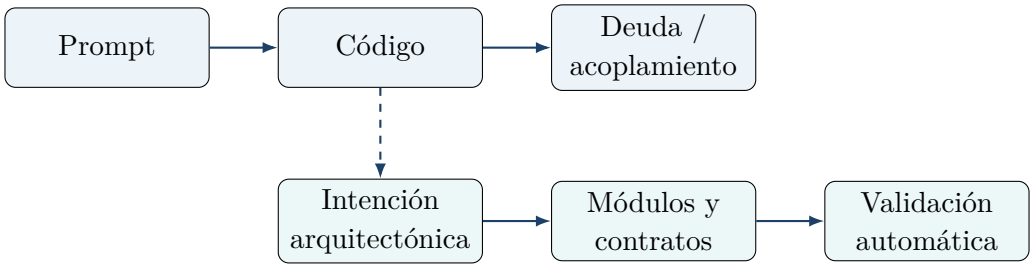
Caso DecidArch

De Vibe Coding a Vibe Architecting

Recuperación arquitectónica mediante modularidad, información oculta, refactoring y validación automática

Caso evaluativo: FlowNow

Un producto de software construido aceleradamente mediante asistentes de IA funciona, pero su evolución está limitada por responsabilidades mezcladas, acoplamiento, complejidad, propiedad difusa de datos, deuda técnica autoadmitida y decisiones específicas de proveedores distribuidas por el código.



Arquitectura de Software – Mantenibilidad – Evolución – Interoperabilidad – Calidad

Índice

1. Propósito de la guía **3**

2. Del Vibe Coding al Vibe Architecting **3**
2.1. Qué entendemos por Vibe Coding en este caso . . . 3
2.2. El giro hacia Vibe Architecting . . . 4

3. Principio rector: Parnas e Information Hiding **4**
3.1. Contrato conceptual de módulo . . . 5

4. Separación de responsabilidades, cohesión y acoplamiento **5**
4.1. Responsabilidad antes que clase . . . 5
4.2. Alta cohesión . . . 6
4.3. Bajo acoplamiento . . . 6

5. Complejidad acotada y granularidad **6**

6. Propiedad del dato e interoperabilidad **7**
6.1. Ownership . . . 7
6.2. Interoperabilidad . . . 8

7. Deuda técnica autoadmitida: detectar no es suficiente **8**
7.1. Regla de la actividad . . . 8

8. Problemas de código que deben buscarse **9**

9. SOLID: principio aplicado, no checklist **9**

10. Patrones de diseño: respuesta a problemas recurrentes **10**

11. Refactoring como mecanismo seguro de recuperación **10**
11.1. Refactoring no es reescritura . . . 11

12. Validación automática: del principio a la fitness function **11**
12.1. Pipeline conceptual . . . 11
12.2. Fitness functions mínimas . . . 11

13. Contrato de intención para el agente de IA **12**

14. Caso evaluativo: FlowNow **13**
14.1. Contexto . . . 13
14.2. Síntomas entregados al equipo . . . 13
14.3. Restricción de negocio . . . 14
14.4. Drivers arquitectónicos . . . 14

15. Metodología de resolución en 10 pasos **14**

16. Consigna evaluativa **16**
16.1. Pregunta central . . . 16
16.2. Condiciones obligatorias . . . 16
16.3. Configuración sugerida . . . 16

17. Entregables **16**
17.1. E1. Diagnóstico AS-IS . . . 16
17.2. E2. Arquitectura TO-BE basada en Parnas . . . 17
17.3. E3. Plan e implementación de refactoring . . . 17
17.4. E4. Fitness Functions y Quality Gate . . . 17
17.5. E5. Gestión de SATD . . . 17
17.6. E6. ADRs . . . 18
17.7. E7. Architectural Intent Prompt . . . 18
17.8. E8. Informe de evidencia AS-IS / TO-BE . . . 18

18. Rúbrica de evaluación **18**
18.1. Niveles de logro . . . 19

19. Criterios de suficiencia de evidencia **19**

20. Preguntas de defensa oral **19**

A. Plantilla: Module Contract **20**

B. Plantilla: Debt Ledger **20**

C. Plantilla breve de ADR **20**

D. Checklist para Pull Request **21**

E. Ejemplo de quality gate docente **21**

1. Propósito de la guía

El propósito de esta guía es que el estudiante aprenda a intervenir una base de código generada o extendida rápidamente con asistentes de inteligencia artificial sin caer en dos extremos:

a) aceptar que "si funciona, está bien diseñado"; o

b) proponer una reescritura completa que elimina aprendizaje, continuidad y evidencia.

La actividad utiliza el término **Vibe Architecting** como una práctica emergente que propone trasladar la atención desde el simple prompting de implementación hacia la **intención arquitectónica, las restricciones, los patrones, las fronteras y la revisión sistemática**. Al no constituir todavía un estándar académico consolidado, la guía lo fundamenta en principios clásicos y verificables de arquitectura y diseño: modularidad e información oculta de Parnas, cohesión y acoplamiento, SOLID, patrones de diseño, refactoring disciplinado, deuda técnica autoadmitida, ownership de datos, interoperabilidad y fitness functions.

Idea central

$$\text{IA que genera código rápidamente} \neq \text{IA que decide correctamente la arquitectura}$$

El objetivo del ejercicio es que la arquitectura pase a ser una **intención explícita y verificable** dentro de la cual el agente puede proponer implementación.

Resultados de aprendizaje

Al finalizar el caso, el estudiante debería ser capaz de:

1. diagnosticar síntomas de erosión arquitectónica y deuda técnica en código generado o modificado con IA;
2. aplicar el criterio de Parnas para descomponer un sistema alrededor de decisiones que deben ocultarse;
3. proponer responsabilidades con alta cohesión y dependencias de bajo acoplamiento;
4. justificar una granularidad modular que no traslade la complejidad interna hacia complejidad de coordinación;
5. declarar ownership de datos y contratos de interoperabilidad;
6. detectar y gestionar *Self-Admitted Technical Debt* sin confundir "eliminar el comentario" con "eliminar la deuda";
7. usar SOLID y patrones sólo cuando resuelvan una presión de cambio concreta;
8. diseñar una estrategia incremental de refactoring con pruebas que preserve comportamiento;
9. transformar reglas arquitectónicas en controles automáticos de CI/CD;
10. redactar ADRs y un contrato de intención arquitectónica para orientar agentes de coding.

2. Del Vibe Coding al Vibe Architecting

2.1. Qué entendemos por Vibe Coding en este caso

En esta actividad, *vibe coding* describe una forma de desarrollo en la que un humano expresa iterativamente intenciones locales mediante lenguaje natural y acepta o ajusta código generado por

un agente. El problema no es la asistencia de IA en sí misma, sino que decisiones estructurales pueden emerger de manera accidental:

$$Prompt_1 \rightarrow Code_1 \rightarrow Prompt_2 \rightarrow Code_2 \rightarrow \cdots$$

Cada respuesta puede ser localmente razonable y, sin embargo, la composición acumulativa puede producir:

$$Coupling \uparrow, \qquad Cohesion \downarrow, \qquad Complexity \uparrow$$

$$DataOwnership \downarrow, \qquad ChangePropagation \uparrow .$$

2.2. El giro hacia Vibe Architecting

El enfoque de Vibe Architecting utilizado pedagógicamente en esta guía cambia el orden del razonamiento:

$$Intent \rightarrow Architecture \rightarrow Constraints \rightarrow Agent \rightarrow Code \rightarrow Validation \rightarrow Feedback.$$

Principio de trabajo

El agente puede generar muchas implementaciones posibles. El equipo debe declarar **qué responsabilidades existen, qué decisiones se ocultan, qué dependencias son permitidas, quién posee los datos, qué invariantes deben mantenerse y cómo será validado el resultado**.

La investigación en architecting colaborativo humano–LLM ya propone separar actividades de **análisis**, **síntesis** y **evaluación** arquitectónica. Esta guía reutiliza esa separación: primero comprender drivers y restricciones, luego proponer una estructura, y finalmente verificarla con escenarios, pruebas y reglas automáticas.

3. Principio rector: Parnas e Information Hiding

D. L. Parnas mostró que la modularización no es valiosa por el mero hecho de "tener módulos". La efectividad depende del **criterio de descomposición**. Su propuesta busca mejorar flexibilidad y comprensibilidad mediante módulos que ocultan decisiones de diseño susceptibles de cambio [1].

Pregunta de Parnas para esta actividad

¿Qué decisión difícil o susceptible de cambio debe quedar oculta dentro de este módulo?

Una descomposición superficial como:

```
controllers/
services/
repositories/
utils/
```

puede ordenar físicamente el código, pero no necesariamente localizar cambios. Si el proveedor de pagos aparece en controladores, servicios, repositorios, DTOs y reportes, el cambio sigue propagándose.

En cambio, un módulo `Payment` puede ocultar:

- proveedor externo;
- tokenización;
- política de reintentos;
- estados internos de transacción;
- persistencia;
- mapeo de errores externos;
- idiosincrasias de una API específica.

El resto del sistema debería depender de un contrato estable, no de esos detalles.

3.1. Contrato conceptual de módulo

Para esta actividad definiremos:

$$M_i = \langle Responsibility,\ Secret,\ Interface,\ OwnedData,\ Dependencies,\ Invariants \rangle.$$

Cada módulo propuesto deberá poder explicar:

1. **Responsibility:** qué problema resuelve.
2. **Secret:** qué decisión de diseño oculta.
3. **Interface:** qué expone a otros módulos.
4. **OwnedData:** qué información gobierna.
5. **Dependencies:** de qué contratos depende.
6. **Invariants:** qué reglas nunca deben romperse.

4. Separación de responsabilidades, cohesión y acoplamiento

4.1. Responsabilidad antes que clase

La primera tarea no consiste en reorganizar clases existentes, sino en descubrir responsabilidades coherentes. Para el caso FlowNow pueden aparecer, por ejemplo:

$$R = \{Identity,\ Customer,\ Request,\ Payment,\ Notification,\ Reporting,\ Integration\}.$$

Esta lista es deliberadamente una **hipótesis inicial**; los estudiantes pueden proponer otra si está mejor justificada por secretos de diseño, drivers y evidencia.

4.2. Alta cohesión

La cohesión expresa cuánto contribuyen los elementos de un módulo a una responsabilidad común.

Ejemplo problemático:

```
class CustomerService {
    createCustomer();
    processPayment();
    sendEmail();
    exportExcel();
    generateSecurityToken();
}
```

La pregunta clave es:

¿Cuántas razones independientes tiene este módulo para cambiar?

Si un mismo módulo cambia por reglas de cliente, pagos, correo, reportes y autenticación, la cohesión es baja.

4.3. Bajo acoplamiento

Bajo acoplamiento no significa ausencia de colaboración. Significa que la colaboración ocurre a través de contratos con una superficie de dependencia controlada.

Se recomienda representar los módulos como un grafo:

$$G = (V, E),$$

donde V son módulos y E dependencias. El equipo deberá observar al menos:

- dependencias entrantes y salientes;
- ciclos;
- dependencias transversales;
- acceso directo a infraestructura ajena;
- propagación esperada de un cambio.

Prueba de cambio

Si mañana cambia el proveedor de pago, ¿cuántos módulos deben modificarse?

$$ChangeImpact(ProviderChange) \approx 1\ module$$

no es una ley matemática, pero sí una expectativa razonable si la decisión del proveedor fue correctamente ocultada.

5. Complejidad acotada y granularidad

Modularizar no significa crear la mayor cantidad posible de módulos. Existen dos fuentes de complejidad:

$$C_{total} = C_{internal} + C_{coordination}.$$

Un módulo enorme aumenta $C_{internal}$; cientos de módulos diminutos pueden aumentar $C_{coordination}$.

Cada propuesta deberá buscar un equilibrio:

$$\boxed{ModuleGranularity = Balance(InternalComplexity, CoordinationComplexity)}$$

Un módulo razonable debería poseer:

- responsabilidad comprensible;
- interfaz limitada y estable;
- decisiones internas ocultables;
- dependencias justificadas;
- complejidad cognitiva administrable;
- tamaño compatible con ownership y evolución.

Anti-patrón evaluativo

No se premiará una propuesta por "tener más microservicios", "más interfaces" o "más capas". La granularidad debe justificarse por responsabilidades, cambio, datos, riesgo y coordinación.

6. Propiedad del dato e interoperabilidad

Uno de los síntomas del caso es que múltiples componentes leen y escriben directamente las mismas tablas. Esto difumina autoridad semántica y hace difícil evolucionar contratos.

6.1. Ownership

La regla pedagógica será:

$$\boxed{\textit{One authoritative owner per datum}}$$

Esto no obliga a tener una base física por módulo. Significa que debe existir una autoridad clara sobre:

- significado;
- validación;
- mutación;
- reglas de consistencia;
- evolución del esquema.

Ejemplo:

Dato	Owner	Lectura por otros	Escritura
Customer	Customer	API / proyección	Customer
Payment	Payment	API / evento	Payment
Request	Request	API / evento	Request
Notification	Notification	auditoría / consulta	Notification

6.2. Interoperabilidad

Una frontera modular no se completa sólo separando carpetas. Debe existir un contrato explícito:

$$Module_A \not\to InternalData_{Module_B}$$

$$Module_A \to Contract_B.$$

Los estudiantes deberán definir, según corresponda:

- API;
- interfaz;
- evento;
- DTO;
- schema;
- política de versionado;
- compatibilidad hacia atrás;
- manejo de cambios incompatibles.

7. Deuda técnica autoadmitida: detectar no es suficiente

La *Self-Admitted Technical Debt* corresponde a deuda que los propios desarrolladores reconocen explícitamente, a menudo mediante comentarios. La literatura ha identificado categorías como deuda de diseño, defectos, documentación, requisitos y pruebas [5, 6].

Ejemplos típicos:

```
TODO: improve this later
FIXME: remove when API changes
HACK: temporary workaround
TEMP: hard-coded until migration
```

7.1. Regla de la actividad

No se evaluará simplemente:

$$SATDComments = 0.$$

Porque borrar comentarios puede ocultar deuda. La condición será:

$$\boxed{UnmanagedSATD = 0}$$

para el alcance intervenido.

Una deuda aceptada excepcionalmente debe transformarse en:

$$SATD \rightarrow Issue \rightarrow Owner \rightarrow Rationale \rightarrow DueDate \rightarrow Verification.$$

Importante

"No quedan TODO" no es evidencia suficiente. El equipo deberá demostrar que la condición señalada fue corregida, transformada en una decisión consciente o registrada como deuda gestionada.

8. Problemas de código que deben buscarse

El diagnóstico inicial deberá considerar al menos las siguientes familias:

Síntoma	Riesgo	Pregunta arquitectónica
Valores duros	cambio costoso, ambiente acoplado	¿es configuración, política o secreto?
Referencias concretas a proveedores	propagación de cambios	¿qué decisión volátil debe ocultarse?
Clases grandes	múltiples razones de cambio	¿qué responsabilidades están mezcladas?
Funciones largas	complejidad cognitiva	¿qué comportamiento puede extraerse?
Acceso compartido a datos	ownership ambiguo	¿quién gobierna esta información?
Dependencias circulares	evolución bloqueada	¿qué dirección de dependencia corresponde?
Duplicación	correcciones inconsistentes	¿existe una abstracción real o sólo repetición?
Interfaces triviales	complejidad accidental	¿qué variabilidad justifica la abstracción?
Uso incorrecto de SOLID	diseño ritualista	¿qué presión de cambio resuelve?
Ausencia de patrón recurrente	solución ad-hoc repetida	¿existe un problema recurrente estable?
SATD	deuda explícita sin gestión	¿corregir, aceptar, registrar o eliminar?
Código externo filtrado al dominio	lock-in conceptual	¿necesitamos adapter / ACL / boundary?

9. SOLID: principio aplicado, no checklist

Los principios SOLID pueden ayudar a localizar responsabilidades y dependencias, pero no deben utilizarse como lista de cumplimiento mecánico.

Regla evaluativa

Cada uso de SOLID deberá responder:
¿Qué problema de cambio, responsabilidad o dependencia estamos resolviendo?

Ejemplos:

- **SRP:** separar reglas de pago de notificaciones porque cambian por causas distintas.
- **OCP:** incorporar nuevos proveedores sin editar políticas de negocio estables.
- **LSP:** evitar sustituciones nominales que cambien semántica observable.
- **ISP:** impedir contratos gigantes que obligan a depender de operaciones irrelevantes.
- **DIP:** hacer que políticas estables dependan de abstracciones y que detalles volátiles implementen esos contratos.

Abstracción innecesaria

Una interfaz con una sola implementación no es automáticamente mala; tampoco es automáticamente buena. Su existencia debe justificarse por frontera, prueba, variabilidad o independencia de políticas. Crear abstracciones "porque SOLID lo dice" añade complejidad accidental.

10. Patrones de diseño: respuesta a problemas recurrentes

Los patrones de diseño documentan soluciones reutilizables a problemas recurrentes de diseño [3]. En este caso no se premiará la cantidad de patrones utilizados.

Problema	Patrón candidato	Criterio de uso
Múltiples proveedores de pago	Strategy / Adapter	variar proveedor sin propagar detalles
APIs externas incompatibles	Adapter / Anti-Corruption Layer	proteger modelo interno
Canales de notificación variables	Strategy	cambiar algoritmo/canal
Creación compleja	Factory	centralizar creación con variabilidad real
Eventos a múltiples consumidores	Observer / Pub-Sub	desacoplar productores y consumidores
Composición de políticas	Decorator / Chain	combinar comportamiento sin gran condicional

El patrón debe surgir después del problema, no antes:

$$RecurringProblem \rightarrow PatternCandidate \rightarrow TradeoffEvaluation.$$

11. Refactoring como mecanismo seguro de recuperación

Fowler define refactoring como una técnica disciplinada para modificar la estructura interna sin cambiar el comportamiento observable [2]. La actividad exige cambios pequeños, pruebas y evidencia

incremental.

Ejemplos de transformaciones:

$$LargeClass \rightarrow ExtractClass$$

$$LongMethod \rightarrow ExtractMethod$$

$$HardcodedValue \rightarrow Configuration$$

$$PrimitiveObsession \rightarrow ValueObject$$

$$DirectVendorDependency \rightarrow Interface + Adapter$$

$$SharedDataMutation \rightarrow OwnershipBoundary$$

11.1. Refactoring no es reescritura

Restricción

No se permite una reescritura completa. El equipo debe demostrar una estrategia incremental, preservando comportamiento y reduciendo riesgo.

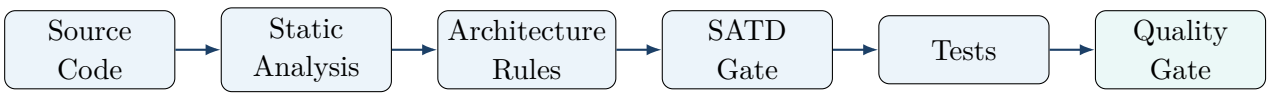
El orden recomendado es:

$$CharacterizationTests \rightarrow SmallRefactoring \rightarrow Tests \rightarrow ArchitectureCheck \rightarrow Commit.$$

12. Validación automática: del principio a la fitness function

Una decisión arquitectónica no está completamente protegida si sólo existe en un documento. El equipo deberá convertir parte de la intención en controles automáticos.

12.1. Pipeline conceptual



12.2. Fitness functions mínimas

La implementación concreta depende del lenguaje, pero la propuesta deberá incluir reglas equivalentes a:

1. no existen ciclos entre módulos;

2. sólo el owner autorizado modifica determinado dato;
3. detalles de proveedor externo no atraviesan la frontera del adapter;
4. no existe SATD no gestionada en código nuevo o intervenido;
5. valores de ambiente no quedan hard-coded;
6. dependencias prohibidas bloquean el merge;
7. contratos y pruebas críticas continúan pasando;
8. reglas arquitectónicas pueden ejecutarse en CI.

Herramientas posibles

El equipo puede utilizar herramientas equivalentes según stack y lenguaje. Algunos ejemplos son:

- **SonarQube / analizadores equivalentes:** smells, complejidad, mantenibilidad y quality gates;
- **Semgrep:** reglas estructurales propias y políticas de código;
- **CodeQL:** consultas sobre seguridad, correctness, maintainability o data flow;
- **ArchUnit, jQAssistant, dependency-cruiser u otros:** reglas de dependencia y arquitectura;
- **frameworks de testing y cobertura:** caracterización, regresión y contratos.

No se exige usar todas. Se evalúa que la selección sea coherente con los riesgos detectados.

13. Contrato de intención para el agente de IA

Antes de pedir código a un agente, el equipo deberá proporcionarle una intención arquitectónica estructurada.

```
ARCHITECTURAL INTENT

Module:
Payment

Responsibility:
Manage the payment lifecycle.

Secret:
Provider-specific payment execution and retry policy.

Owned data:
Payment, Transaction.

Allowed dependencies:
CustomerIdentity API.

Forbidden dependencies:
Customer database.
Notification implementation.
Reporting database.

Public contract:
createPayment()
```

```
authorizePayment()
capturePayment()
refundPayment()
getPaymentStatus()

Quality constraints:
- no circular dependency
- no unmanaged SATD
- provider configuration externalized
- provider types must not leak outside adapter
- behavior preserved by tests

Evidence:
unit tests + contract tests + architecture rule + ADR
```

Vibe Architecting en una frase

El prompt deja de pedir "haz que funcione" y comienza a declarar **responsabilidad, secreto, frontera, datos, restricciones y evidencia**.

14. Caso evaluativo: FlowNow

14.1. Contexto

FlowNow es una plataforma digital para gestionar:

- clientes;
- solicitudes;
- pagos;
- notificaciones;
- integraciones externas;
- reportes.

La empresa buscó acelerar el time-to-market mediante asistentes de IA. Durante seis meses, distintos desarrolladores y agentes extendieron el producto con prompts locales. La velocidad inicial fue alta y las funcionalidades principales funcionan, pero la organización observa que cambios pequeños requieren modificaciones crecientes en archivos no relacionados.

14.2. Síntomas entregados al equipo

El repositorio contiene evidencia de:

- lógica de negocio en controladores;
- clases y métodos excesivamente grandes;
- utilitarios genéricos con responsabilidades mezcladas;
- valores hard-coded;
- URLs, nombres de proveedores y flags distribuidos;
- acceso directo a tablas desde varios componentes;

- dependencias circulares;
- duplicación;
- interfaces sin variabilidad real;
- uso inconsistente de SOLID;
- ausencia de fronteras claras para APIs de terceros;
- comentarios TODO, FIXME, HACK, TEMP y WORKAROUND;
- pruebas insuficientes alrededor de comportamiento crítico.

14.3. Restricción de negocio

La gerencia establece:

> "No queremos detener el producto ni reescribirlo. Queremos continuar usando IA para acelerar el desarrollo, pero necesitamos que el sistema pueda evolucionar sin que cada cambio aumente exponencialmente el costo y el riesgo."

14.4. Drivers arquitectónicos

Los drivers prioritarios del caso serán:

Driver	**Interpretación operacional**
Maintainability	comprender y modificar una responsabilidad sin explorar el sistema completo
Modifiability	localizar el impacto de cambios previsibles
Evolvability	incorporar nuevos proveedores y capacidades sin reestructuración masiva
Interoperability	integrar mediante contratos explícitos y versionables
Testability	preservar comportamiento durante recuperación incremental
Data Ownership	conocer autoridad semántica y de mutación
Bounded Complexity	evitar módulos gigantes y micro-fragmentación
Traceability	relacionar problema, decisión, refactoring y evidencia

15. Metodología de resolución en 10 pasos

Paso 1. Construir la línea base

Recolectar evidencia antes de intervenir:

- estructura actual;
- dependencias;
- complejidad;
- smells;
- SATD;
- ownership de datos;
- cobertura y pruebas;

- puntos de integración.

El resultado debe ser reproducible: scripts, reportes o comandos.

Paso 2. Formular escenarios de cambio

Definir al menos cuatro escenarios, por ejemplo:

1. cambiar proveedor de pagos;
2. agregar un nuevo canal de notificación;
3. cambiar el modelo de Customer;
4. introducir una nueva integración externa;
5. separar reporting de datos transaccionales.

Cada escenario debe mostrar el impacto AS-IS.

Paso 3. Descubrir responsabilidades

Agrupar comportamiento por responsabilidad y razón de cambio. No copiar la estructura actual.

Paso 4. Identificar secretos de Parnas

Por cada módulo candidato, declarar la decisión que debe quedar oculta.

Paso 5. Diseñar contratos y ownership

Definir API/interfaz/eventos y autoridad sobre datos.

Paso 6. Evaluar cohesión, acoplamiento y granularidad

Construir un grafo TO-BE y justificar fronteras.

Paso 7. Diseñar el plan incremental de refactoring

Ordenar cambios para reducir riesgo. Priorizar seams, pruebas, extracción de dependencias y fronteras.

Paso 8. Implementar un slice representativo

No es obligatorio refactorizar el sistema completo. Se exige intervenir un slice suficiente para demostrar la estrategia.

Paso 9. Automatizar restricciones

Implementar fitness functions y quality gates.

Paso 10. Comparar evidencia AS-IS vs TO-BE

La conclusión debe basarse en medidas y escenarios de cambio, no sólo en percepción estética.

16. Consigna evaluativa

Desafío

Diseñe y demuestre una estrategia para transformar una base de código deteriorada por desarrollo incremental asistido por IA en una arquitectura modular, mantenible y evolutiva, sin realizar una reescritura completa y preservando el comportamiento funcional relevante.

16.1. Pregunta central

¿Qué evidencia demuestra que el sistema ahora puede cambiar sin propagar innecesariamente el impacto de ese cambio?

16.2. Condiciones obligatorias

La solución debe:

1. aplicar explícitamente el criterio de Parnas;
2. separar responsabilidades justificando cohesión y acoplamiento;
3. declarar ownership de datos;
4. proteger interoperabilidad mediante contratos;
5. gestionar SATD sin ocultarla;
6. utilizar refactoring incremental con pruebas;
7. convertir al menos tres decisiones arquitectónicas en reglas automatizadas;
8. documentar decisiones mediante ADR;
9. incluir instrucciones de Vibe Architecting para un agente de coding;
10. demostrar cambios mediante evidencia AS-IS / TO-BE.

16.3. Configuración sugerida

- Equipos: 3–4 estudiantes.
- Duración: 2 semanas.
- Trabajo estimado: 10–14 horas por equipo, ajustable por el docente.
- Tecnologías: libres, de acuerdo con el repositorio entregado.
- Uso de IA: permitido y esperado, pero debe quedar trazabilidad de cómo fue controlada por intención arquitectónica.

17. Entregables

17.1. E1. Diagnóstico AS-IS

Máximo sugerido: 5 páginas más anexos automáticos.

Debe incluir:

- mapa de módulos/clases actuales;
- smells y complejidad;
- SATD clasificada;
- ciclos y dependencias problemáticas;
- ownership de datos actual;
- cuatro escenarios de cambio y su impacto;
- baseline reproducible.

17.2. E2. Arquitectura TO-BE basada en Parnas

Debe incluir:

- módulos propuestos;
- responsabilidad;
- secreto de información;
- contratos;
- owned data;
- dependencias permitidas/prohibidas;
- invariantes;
- justificación de granularidad.

17.3. E3. Plan e implementación de refactoring

Debe presentar:

- secuencia incremental;
- pruebas de caracterización;
- cambios realizados;
- pull request o diff;
- evidencias de comportamiento preservado.

17.4. E4. Fitness Functions y Quality Gate

Al menos tres reglas arquitectónicas automatizadas y un pipeline reproducible.

17.5. E5. Gestión de SATD

Inventario inicial, clasificación, resolución/aceptación y estado final. La meta del alcance intervenido es:

$$UnmanagedSATD = 0.$$

17.6. E6. ADRs

Mínimo 5 ADRs breves que documenten decisiones significativas.

Ejemplos:

- límite del módulo Payment;
- propiedad de Customer;
- estrategia para proveedores externos;
- forma de integración entre módulos;
- política para SATD y quality gates.

17.7. E7. Architectural Intent Prompt

Un contrato reusable que permita a otro agente de IA implementar una nueva capacidad sin romper las fronteras propuestas.

17.8. E8. Informe de evidencia AS-IS / TO-BE

Debe usar la matriz siguiente y añadir métricas específicas del stack.

Indicador	**AS-IS**	**TO-BE**
SATD no gestionada	X	0
Ciclos entre módulos	X	0
Dependencias prohibidas	X	0
Hard-coded críticos	X	0 o justificados
Datos sin owner	X	0
Complejidad extrema	X	$\downarrow$
Duplicación relevante	X	$\downarrow$
Cobertura crítica	X %	$\uparrow$
Cambio de proveedor afecta	N módulos	acotado
ADRs	0/X	≥ 5
Fitness functions	0/X	≥ 3

18. Rúbrica de evaluación

Dimensión	**Peso**	**Evidencia esperada**
Diagnóstico fundamentado	10 %	baseline reproducible, smells, SATD, dependencias y escenarios de cambio
Descomposición por Parnas	20 %	módulos definidos por secretos/decisiones de cambio, no sólo capas técnicas
Cohesión, acoplamiento y complejidad	15 %	responsabilidades claras, grafo, ciclos, granularidad y reducción de propagación
Data ownership e interoperabilidad	15 %	autoridad explícita, contratos, versionado y ausencia de acceso interno injustificado
SATD y calidad de código	10 %	clasificación, resolución o gestión explícita; no ocultamiento de deuda
Refactoring y pruebas	15 %	pasos pequeños, comportamiento preservado, evidencia automática

Fitness functions / quality gates	10 %	restricciones arquitectónicas ejecutables en CI
ADRs y Vibe Architecting	5 %	decisiones trazables e instrucciones útiles para agentes
Total	**100 %**	

18.1. Niveles de logro

Nivel	Descriptor
Excelente	La solución localiza decisiones de cambio, demuestra reducción del impacto, automatiza invariantes y utiliza IA dentro de fronteras verificables. Existe coherencia entre drivers, módulos, código, pruebas y ADRs.
Competente	La arquitectura es razonable y está respaldada por evidencia, aunque algunas reglas o límites podrían estar mejor automatizados o justificados.
En desarrollo	Existen mejoras de código, pero la descomposición sigue principalmente capas técnicas o convenciones; la evidencia de evolvability es parcial.
Insuficiente	Predomina limpieza superficial, reordenamiento de carpetas o uso declarativo de SOLID/patrones sin mostrar cómo se reducen cambios, dependencias o deuda.

19. Criterios de suficiencia de evidencia

Una propuesta no se considerará demostrada sólo por diagramas. Debe existir triangulación entre:

$$ArchitectureModel + SourceCode + AutomatedEvidence.$$

Para cada afirmación relevante, el equipo debe poder responder:

1. ¿qué problema observamos?;
2. ¿qué decisión tomamos?;
3. ¿qué artefacto cambió?;
4. ¿qué prueba/regla demuestra el resultado?;
5. ¿qué escenario de cambio mejoró?;

Regla de evidencia

Una afirmación como “ahora el sistema está desacoplado” no es suficiente. Debe mostrarse, por ejemplo, que una sustitución de proveedor dejó de modificar cinco componentes y ahora se concentra en un adapter, con una regla automática que impide filtrar tipos externos.

20. Preguntas de defensa oral

El docente podrá seleccionar preguntas como:

1. ¿Cuál es el secreto más importante ocultado por cada módulo?

2. ¿Qué módulo eliminaría si tuviera que simplificar la propuesta?
3. ¿Qué dependencia sigue siendo riesgosa y por qué?
4. ¿Qué deuda decidieron aceptar conscientemente?
5. ¿Dónde una interfaz agregaría complejidad sin beneficio?
6. ¿Qué patrón rechazaron y por qué?
7. ¿Cómo saben que un refactoring preservó comportamiento?
8. ¿Qué fitness function protege la decisión más importante?
9. ¿Qué ocurriría si cambia el modelo de datos de Customer?
10. ¿Cómo impedirían que un agente de IA vuelva a introducir el problema original?

A. Plantilla: Module Contract

Campo	Contenido
Module	
Responsibility	
Information secret	
Reason to change	
Owned data	
Public interface	
Incoming dependencies	
Outgoing dependencies	
Forbidden dependencies	
Invariants	
Quality drivers	
SATD status	
Tests / evidence	
ADR reference	

B. Plantilla: Debt Ledger

ID	Tipo	Evidencia / ubicación	Decisión	Owner / fecha
TD-001	Design		Refactor / Accept / Remove	
TD-002	Defect			
TD-003	Test			

C. Plantilla breve de ADR

```
# ADR-XXX: <Decision title>

Status:
Context:
Architectural concern:
```

```
Decision:
Alternatives considered:
Consequences:
Trade-offs:
Evidence / fitness function:
Affected modules:
Date:
```

D. Checklist para Pull Request

- ☐ El comportamiento observable relevante continúa pasando sus pruebas.
- ☐ La responsabilidad modificada está explícitamente declarada.
- ☐ El secreto del módulo no se filtra.
- ☐ No se introducen dependencias prohibidas.
- ☐ Los datos modificados tienen owner claro.
- ☐ No aparece SATD no gestionada.
- ☐ Configuración y valores variables no quedan hard-coded.
- ☐ No se crea una abstracción sin presión de cambio o frontera justificable.
- ☐ Los ADRs afectados fueron actualizados.
- ☐ Las fitness functions relevantes pasan en CI.
- ☐ El prompt/agente utilizado respetó el Architectural Intent.

E. Ejemplo de quality gate docente

Los siguientes umbrales son **políticas del ejercicio**, no valores universales de ingeniería:

Regla	**Condición del ejercicio**
Unmanaged SATD en código intervenido	0
Ciclos entre módulos TO-BE	0
Dependencias prohibidas	0
Hard-coded de secretos/URLs de ambiente	0
Fitness functions automatizadas	≥ 3
ADRs	≥ 5
Escenarios de cambio comparados	≥ 4
Pruebas críticas	deben pasar antes y después

Referencias

[1] D. L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM*, 15(12), 1053–1058, 1972. DOI: https://doi.org/10.1145/361598.361623.

[2] M. Fowler. *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley Professional, 2018/2019.

[3] E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[4] R. C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Pearson, 2002.

[5] A. Potdar and E. Shihab. "An Exploratory Study on Self-Admitted Technical Debt." *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 91–100, 2014. DOI: https://doi.org/10.1109/ICSME.2014.31.

[6] E. da S. Maldonado and E. Shihab. "Detecting and Quantifying Different Types of Self-Admitted Technical Debt." *IEEE 7th International Workshop on Managing Technical Debt*, pp. 9–15, 2015. DOI: https://doi.org/10.1109/MTD.2015.7332619.

[7] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M. S. Aktar, T. Mikkonen. "Towards Human-Bot Collaborative Software Architecting with ChatGPT." *EASE 2023*, ACM, 2023. DOI: https://doi.org/10.1145/3593434.3593468.

[8] VibeArchitecting. "From vibe coding to vibe architecting." Recurso de práctica profesional emergente. https://vibearchitecting.com/. Consultado en 2026.

[9] SonarSource. SonarQube Server Documentation: Design and Architecture; maintainability and quality rules. https://docs.sonarsource.com/sonarqube-server/.

[10] Semgrep. Documentation: custom rules and CI integration. https://semgrep.dev/docs/.

[11] GitHub. CodeQL Documentation. https://codeql.github.com/docs/.

Cierre conceptual

La IA puede acelerar la escritura del código. La responsabilidad del arquitecto es hacer explícitas y verificables las decisiones que permiten que ese código continúe siendo comprensible, mantenible y evolutivo.